PHASE 4 · ADVANCED

DAY 23

# Web Requests & APIs

requests · REST · JSON · Bearer/API-Key Auth · httpx (async)

| 2 hrs | 4 | 20 | 3 |
| --- | --- | --- | --- |
| Session Length | Concept Files | Practice Problems | Live API Demos |

30-Day Python Programming Course · Instructor Session Guide

# Session Overview

**Goal:** Students learn to call real-world REST APIs from Python — sending GET/POST requests, reading JSON responses, authenticating with Bearer tokens and API keys, and making concurrent requests with the async httpx client.

**Outcome:** By the end of the session, students can confidently consume any JSON REST API — including ones that require authentication — and can explain when async requests provide a real speed advantage.

## Timeline

| Time | Segment | Deliverable |
|---|---|---|
| 0:00 – 0:10 | Recap Day 22 + intro to REST/APIs | — |
| 0:10 – 0:35 | requests basics: GET, POST, params, headers | day23_01_requests_basics.py |
| 0:35 – 1:00 | REST verbs, status codes, nested JSON, DataFrames | day23_02_rest_api_json.py |
| 1:00 – 1:10 | Break | — |
| 1:10 – 1:35 | Auth: Bearer tokens, API keys, Sessions | day23_03_auth_headers.py |
| 1:35 – 1:55 | httpx async client + asyncio.gather() | day23_04_httpx_async.py |
| 1:55 – 2:00 | Wrap-up, Q&A, Day 24 preview | — |

**TIP: Instructor Note**
All demos in this session hit a local mock API (TiffinExpress) started automatically by day23_00_mock_server.py — no internet dependency, so the session runs identically every time.

# Prerequisites & Preparation

## Requirements

| Requirement | Details |
|---|---|
| Python | 3.12, from Day 1 setup |
| Packages | pip install requests httpx pandas --break-system-packages |
| Prior days | Day 9 (errors), Day 16 (async), Day 22 (Pandas) |
| Files needed | day23_00_mock_server.py must be in the same folder as every other Day 23 file |

## Instructor Pre-Session Checklist

[ ] Confirm requests, httpx, and pandas are installed in the demo environment
[ ] Copy day23_00_mock_server.py into the same folder as all other Day 23 files
[ ] Run each concept file once before class to warm the terminal cache
[ ] Have Day 22 (Pandas) DataFrame examples on hand for the JSON-to-DataFrame section
[ ] Pre-open a terminal split: one for concept files, one for practice problems

**WATCH OUT: Watch Out**
Do not attempt to demo against a public API such as httpbin.org during a live class — rate limiting or an outage can derail the session. Stick to the local mock server.

# Step-by-Step Instruction Guide

## STEP 1 — requests Basics (GET, POST, Params, Headers)

### "What to Say"

*"Every time an app on your phone shows you data — Swiggy showing restaurants, or IRCTC showing train seats — it sent an HTTP request to a server and got JSON back. Today, we write that exact request ourselves, using the requests library."*

```
# Terminal — install requests (once per machine)
pip install requests --break-system-packages

# Run the file
python day23_01_requests_basics.py

Status code: 200
Number of orders: 5
```

| Error | Cause | Fix |
|---|---|---|
| ModuleNotFoundError: requests | Package not installed | pip install requests --break-system-packages |
| ConnectionRefusedError | Mock server not started | Ensure start_server() runs before any request |
| JSONDecodeError | Called .json() on an empty/HTML body | Check .status_code and .text before .json() |

**DONE WHEN: Done When**
Students can call requests.get() with query params, call requests.post() with a JSON body, and correctly read .status_code and .json() from the response.

## STEP 2 — REST Verbs, Status Codes & JSON Parsing

### "What to Say"

*"REST APIs follow a pattern: GET reads, POST creates, PUT updates, DELETE removes. Once you know this pattern, you can guess how almost any REST API works before even reading its docs."*

```
python day23_02_rest_api_json.py

CREATE -> new order id: 6, status: 201
UPDATE -> new status: delivered
DELETE -> {'message': 'Order 6 cancelled successfully'}
```

| Error | Cause | Fix |
|---|---|---|
| KeyError on nested JSON | Wrong key path into a nested dict | Print the dict first, then index step-by-step |

| | | |
|---|---|---|
| TypeError: list indices | Treating a list response like a dict | Check type() before indexing |

**DONE WHEN: Done When**
Students can name which HTTP verb performs each CRUD action and can convert a JSON list response into a Pandas DataFrame.

## STEP 3 — Authentication: Bearer Tokens & API Keys

### "What to Say"

*"Public endpoints are open to everyone, but private data — like your Paytm wallet balance — needs proof of who you are. That proof travels as a header on every request: a Bearer token or an API key."*

```
python day23_03_auth_headers.py

Status code: 401
Error message: Unauthorized. Missing or invalid Bearer token.

headers = {'Authorization': f'Bearer {VALID_TOKEN}'}
Status code: 200
```

| Error | Cause | Fix |
|---|---|---|
| 401 with a token present | Missing 'Bearer ' prefix or wrong header name | Confirm exact header spelling and format |
| Token works once then fails | Reused a Session with stale headers | Update session.headers explicitly |

**DONE WHEN: Done When**
Students can add a Bearer token and a custom API-key header to a request, and can explain why a 401 is different from a 404.

## STEP 4 — httpx & Concurrent Requests

### "What to Say"

*"If you need data from five endpoints, doing them one after another wastes time waiting. httpx plus asyncio.gather lets you fire all five at once and wait for all of them together."*

```
python day23_04_httpx_async.py

Sequential (5 requests, one-by-one): 0.1290s
Concurrent (5 requests, gathered): 0.0405s
Speed-up factor: 3.2x faster
```

| Error | Cause | Fix |
|---|---|---|
| RuntimeWarning: coroutine was never awaited | Forgot `await` or asyncio.run() | Wrap the call in asyncio.run(...) |
| Client already closed error | Used client outside its `async with` block | Keep all requests inside the `async with` scope |

**DONE WHEN: Done When**
Students can write an async function using httpx.AsyncClient and asyncio.gather() to fetch multiple endpoints concurrently.

# Session Wrap-Up

## Student Checklist

[ ] I can send a GET request with query parameters using requests
[ ] I can send a POST request with a JSON body and read the created resource
[ ] I can name the HTTP verb for each CRUD operation
[ ] I can read a 2xx/4xx/5xx status code and know what family it belongs to
[ ] I can add a Bearer token or API key header to an authenticated request
[ ] I can write a basic async function using httpx.AsyncClient + asyncio.gather()

## Key Concepts Recap

| Concept | One-line Summary |
|---|---|
| GET / POST / PUT / DELETE | The four core verbs mapping to Read / Create / Update / Delete |
| Query parameters | params= dict → auto-encoded ?key=value in the URL |
| Status code families | 2xx success, 4xx client error, 5xx server error |
| Bearer token | Authorization: Bearer <token> header for identity-based auth |
| API key | A custom header (e.g. X-API-Key) some APIs prefer instead |
| httpx + asyncio.gather | Run many I/O-bound requests concurrently instead of one-by-one |

## Next Day Preview — Day 24: FastAPI

Tomorrow students flip roles — instead of calling someone else's API, they build their own with FastAPI: routes, path/query parameters, Pydantic validation models, async endpoints, and the auto-generated Swagger UI at /docs.

# Quick Reference Card

## Daily Workflow Commands

```
pip install requests httpx pandas --break-system-packages
python day23_00_mock_server.py     # optional: run server standalone
python day23_01_requests_basics.py
python day23_02_rest_api_json.py
python day23_03_auth_headers.py
python day23_04_httpx_async.py
```

## Tool & Method Summary

| Call | Purpose |
|---|---|
| requests.get(url, params=, headers=) | Read data from a resource |
| requests.post(url, json=) | Create a new resource |
| requests.put(url, json=) | Update an existing resource |
| requests.delete(url) | Remove a resource |
| response.status_code / .ok / .json() | Inspect the response |
| requests.Session() | Reuse headers/connections across many calls |
| httpx.AsyncClient() | Async HTTP client for use inside async def |
| asyncio.gather(*tasks) | Run many coroutines concurrently |